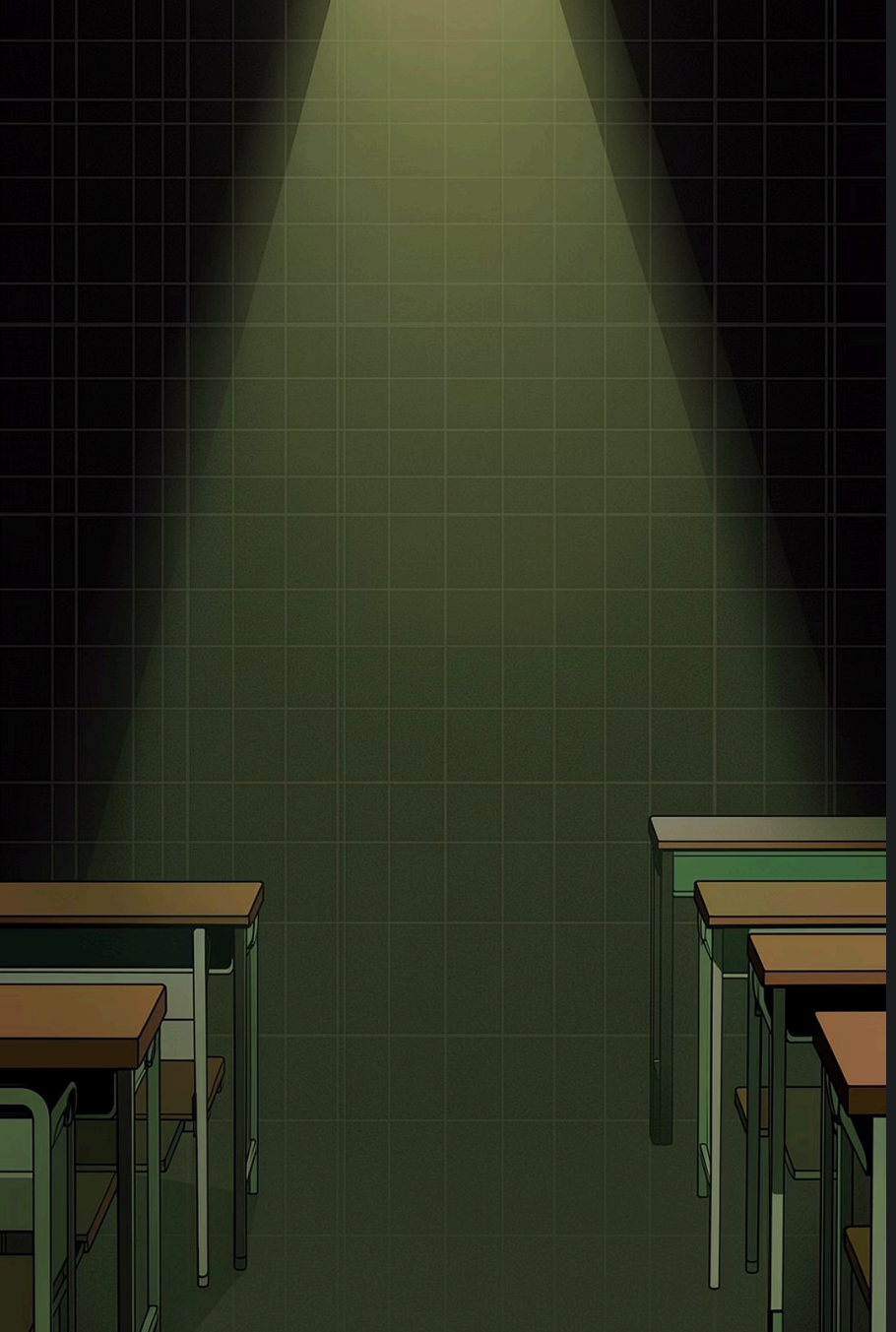


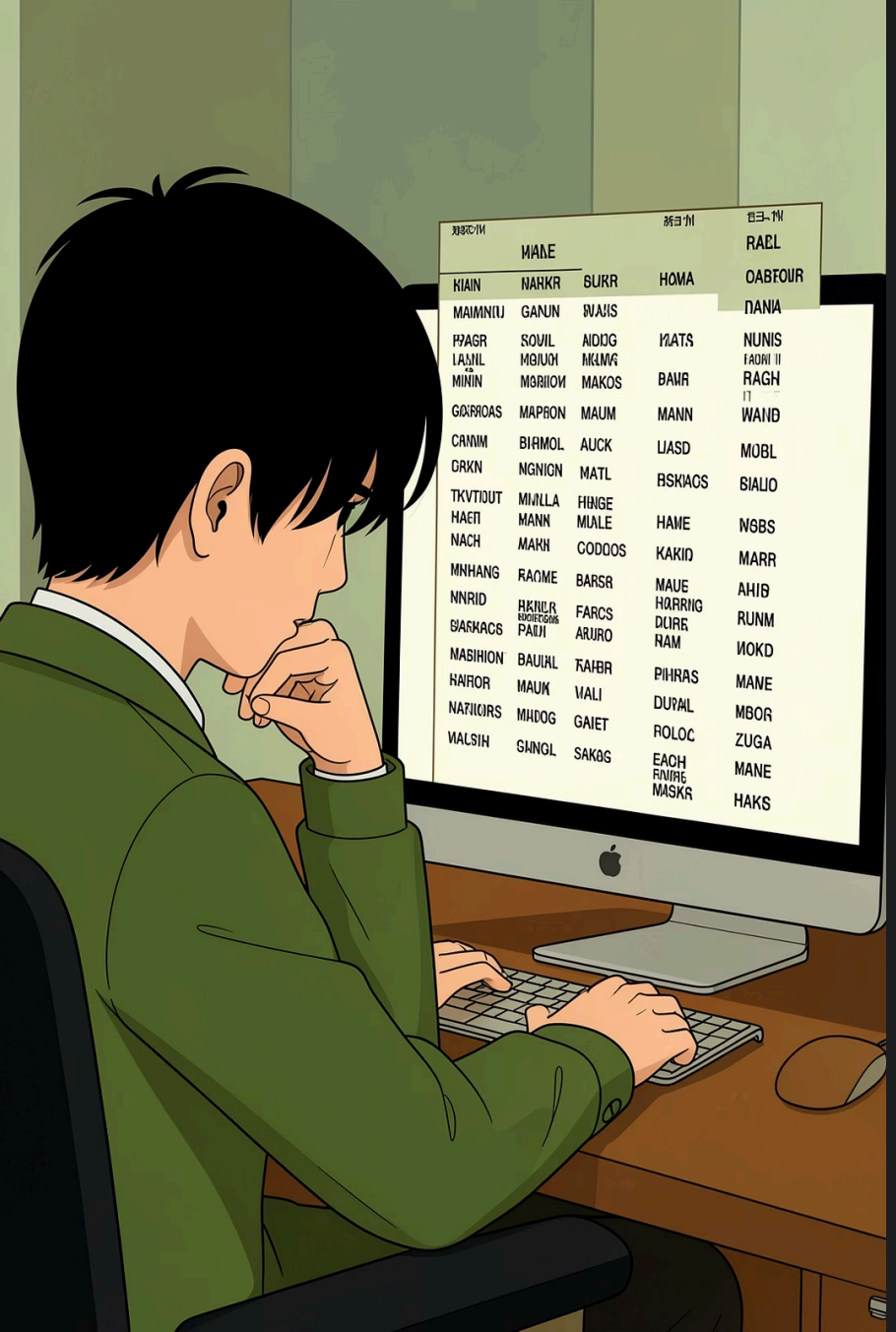
Списки и алгоритмы: Поиск и сортировка

Цели урока: понять основные приёмы поиска и сортировки в списках, уметь реализовать простые алгоритмы на Python и оценивать их эффективность.

9 КЛАСС

PYTHON

Made with GAMMA



Задача

Как быстро найти одно имя в списке из 1000? Два подхода:

Линейный (последовательный) поиск

Перебираем элементы один за другим — O(n) времени. Простая реализация, но медленно для больших списков.

Поиск в отсортированном списке (бинпоиск)

Делим диапазон пополам каждый шаг — O(log n). Требует предварительной сортировки.

Вывод: если список заранее отсортирован, поиск становится заметно быстрее.

Made with GAMMA

Поиск максимума без встроенных функций

Идея: пройти по списку и сохранять текущий максимум.

Код (пошагово)

```
def find_max(lst):
    max_val = lst[0]  # считаем первый элемент текущим максимумом
    for x in lst:     # перебираем все элементы
        if x > max_val:  # нашли больше — обновляем
            max_val = x
    return max_val
```

Трассировка на примере

lst = [2, 7, 4, 9, 3]

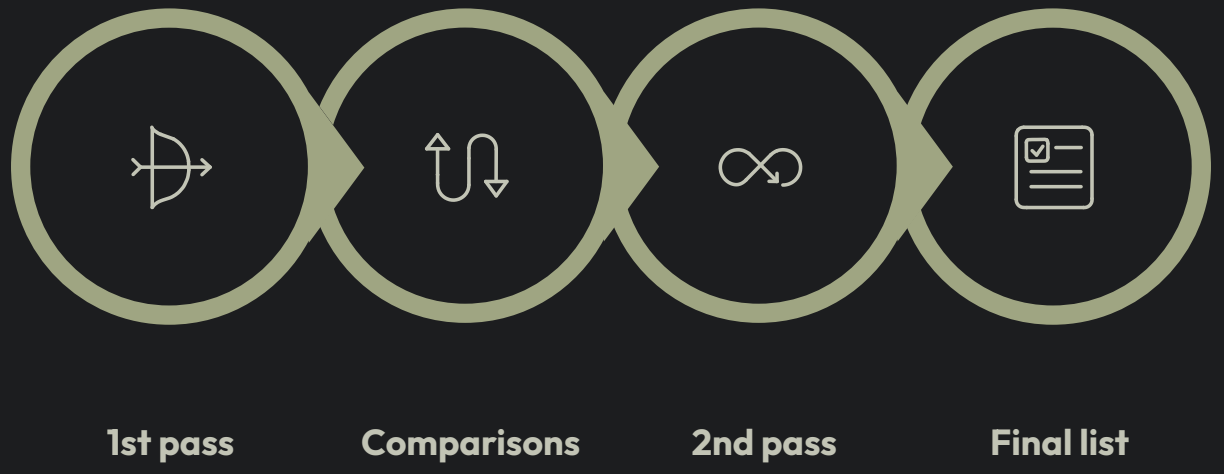
- Начало: max_val = 2
- x=2 → не больше
- x=7 → обновить max_val = 7
- x=4 → нет
- x=9 → обновить max_val = 9
- Возврат 9

Made with GAMMA

Пузырьковая сортировка — идея «всплывания»

Принцип: несколько раз проходить по списку и обменивать соседние элементы, если они в неправильном порядке — большие «всплывают» вправо как пузырьки.

Пример: [5, 3, 8, 1, 4]



После нескольких проходов список станет [1, 3, 4, 5, 8].

Made with GAMMA

Код bubble_sort с пояснениями

Код

```
def bubble_sort(lst):
    n = len(lst)              # длина списка
    for i in range(n):        # повторяем n раз
        swapped = False       # флаг — были ли обмены
        for j in range(0, n - i - 1):  # проходим по неотсортированной части
            if lst[j] > lst[j+1]:      # сосед в неправильном порядке
                lst[j], lst[j+1] = lst[j+1], lst[j]  # меняем местами
                swapped = True
        if not swapped:         # если за проход не было обменов — готово
            break
    return lst
```

Пояснения по строкам

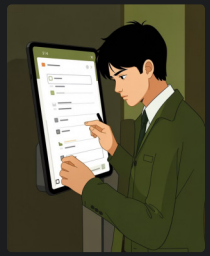
- n = len(lst) — нужна длина для границ циклов.
- Внешний цикл i управляет количеством проходов; каждая итерация ставит один элемент на своё место справа.
- Внутренний цикл j сравнивает соседей; диапазон уменьшается на i, потому что правые элементы уже отсортированы.
- swapped позволяет остановиться раньше, если список уже упорядочен — оптимизация.

Made with GAMMA

Самостоятельные задания — 3 уровня

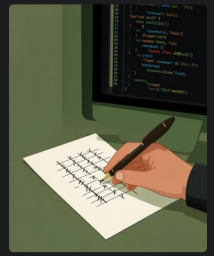
Уровень 1 — Начинающий

Напишите функцию `find_min(lst)`, возвращающую минимум в списке без встроенной min().



Уровень 2 — Средний

Реализуйте `linear_search(lst, target)`, возвращающую индекс первого вхождения или -1. Оцените временную сложность.



Уровень 3 — Продвинутый

Модифицируйте пузырьковую сортировку, чтобы она считала и возвращала количество обменов, выполненных до сортировки.

Подсказка: тестируйте на коротких списках вручную и печатайте промежуточные состояния для отладки.

Made with GAMMA

Шпаргалка

Списки: создание — lst = [1,2,3]; доступ — lst[i]; срезы — lst[a:b]

range(): range(n) — числа 0..n-1; range(a,b) — от a до b-1.

Обмен элементов: lst[i], lst[j] = lst[j], lst[i]

Сложность алгоритма: O(1) — константная; O(n) — линейная; O(n^2) — квадратичная (пузырёк).

Совет: для больших данных выбирайте алгоритмы с меньшей асимптотикой.

Made with GAMMA

Домашнее задание

Напишите функцию `is_sorted(lst)`, которая возвращает True, если список упорядочен по возрастанию, иначе False.

Требования:

1. Не использовать встроенные сортировки или функции сравнения списков.
2. Реализовать за O(n) времени — один проход.
3. Протестировать на примерах: [], [1], [1,2,2,3], [3,2,1].

> Совет: проходите по списку и сравнивайте каждый элемент с предыдущим.

Made with GAMMA